

IDX Exchange – 12 Week NLP Internship

Real Estate Listing Intelligence System

About This Internship Project

This 12-week program guides you through building a production-ready NLP system for real estate listings. You will work with actual MLS data from three SQL tables (rets_property, rets_openhouse, california_sold) and build capabilities for search, extraction, classification, and compliance checking.

Technical Setup

Each intern will:

- Create individual GitHub repository with teammates as collaborators
- Install Docker Desktop for local development environment
- Load three SQL database files into MySQL container
- Work in local environment, pushing code to GitHub weekly
- Use Python 3.11+, standard NLP libraries (nltk, spaCy, transformers)
- Follow test-driven development with pytest

Database Overview

You will work with three main tables:

- rets_property: Active/pending listings with remarks, pricing, features
- rets_openhouse: Open house schedules and details
- california_sold: Historical sold properties with transaction data

Week 0: Environment Setup (Optional Prep Week)

Overview

This optional prep week helps students set up their development environment before Week 1. Students who complete this setup early can dive straight into taxonomy building in Week 1 without losing time to environment issues.

Time Estimate: 3-5 hours (can be done before official start)

Setup Checklist

- ☐ Install Docker Desktop and verify it runs
- ☐ Install Python 3.11+ and verify version
- ☐ Set up Git and create GitHub account
- ☐ Install VS Code or preferred IDE
- ☐ Clone starter repository and verify structure
- ☐ Run 'hello world' test to confirm Python environment
- ☐ Start MySQL container and verify connection

Detailed Setup Steps

1. Install Docker Desktop

- Download from docker.com for your OS (Windows/Mac/Linux)
- Allocate at least 4GB RAM in Docker settings
- Test: Run `docker --version` in terminal
- Expected output: Docker version 20.x or higher

2. Install Python 3.11+

- Download from python.org or use package manager
- Add Python to your PATH during installation
- Test: Run `python --version` or `python3 --version`
- Expected output: Python 3.11.x or higher

3. Set Up Git and GitHub

- Install Git from git-scm.com
- Create GitHub account at github.com (if you don't have one)
- Configure Git with your name and email:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

4. Create Project Structure

- Create your project directory:

```
mkdir -p nlp-internship/{data/{raw,processed,models},scripts,tests,notebooks}
```

```
cd nlp-internship
```

```
git init
```

5. Create docker-compose.yml

Create a file named `docker-compose.yml` in your project root:

```
version: '3.8'
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: real_estate
    ports:
      - "3306:3306"
```

```
volumes:
  - ./data/raw:/docker-entrypoint-initdb.d:ro
  - mysql_data:/var/lib/mysql
volumes:
  mysql_data:
```

6. Install Python Dependencies

Create requirements.txt file:

```
mysql-connector-python==8.0.33
pandas==2.0.3
nltk==3.8.1
spacy==3.6.0
pytest==7.4.0
jupyter==1.0.0
```

Install packages:

```
pip install -r requirements.txt
```

7. Verify Setup

Create tests/test_setup.py to verify everything works:

```
import sys
import mysql.connector
import pandas as pd
import nltk

def test_python_version():
    assert sys.version_info >= (3, 11), "Python 3.11+ required"
    print("✓ Python version OK")

def test_packages():
    print("✓ All required packages installed")

def test_docker():
```

```

try:
    conn = mysql.connector.connect(
        host='localhost',
        user='root',
        password='root',
        database='real_estate'
    )
    conn.close()
    print("✓ MySQL connection successful")
except:
    print("⚠ MySQL not running - start with: docker-compose up -d")

if __name__ == "__main__":
    test_python_version()
    test_packages()
    test_docker()
    print("\nSetup verification complete!")

```

Run the test:

```
python tests/test_setup.py
```

Common Issues and Solutions

Docker won't start:

- Ensure virtualization is enabled in BIOS (Windows)
- Check if another MySQL instance is using port 3306
- Try: docker-compose down then docker-compose up -d

Python version conflicts:

- Use python3 instead of python command
- Consider using pyenv for version management
- Create virtual environment: python3 -m venv venv

Permission errors (Linux/Mac):

- Add user to docker group: `sudo usermod -aG docker $USER`
- Log out and back in for changes to take effect

What If I Get Stuck?

- Post in team Slack channel with error messages
- Schedule office hours with mentor
- Check Docker/Python documentation
- Pair with a teammate who has setup working

Success Criteria

By the end of Week 0, you should be able to:

- ✓ Run `docker-compose up -d` without errors
- ✓ Execute `python --version` and see 3.11+
- ✓ Connect to MySQL container from Python
- ✓ Run `pytest tests/test_setup.py` successfully
- ✓ Push code to GitHub repository

Once you've completed these steps, you're ready for Week 1!

Week 1: Domain Understanding + Taxonomy

Overview

Build a real estate terminology taxonomy and sample dataset. This taxonomy will be the foundation for entity extraction, query parsing, and all downstream NLP work.

Setup Steps

1. Install Docker Desktop (docker.com), allocate 4GB RAM minimum

2. Create project structure:

```
mkdir -p nlp-internship/{data/{raw,processed,models},scripts,tests,notebooks}
cd nlp-internship
```

3. Create docker-compose.yml:

```
version: '3.8'
services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: real_estate
    ports:
      - "3306:3306"
    volumes:
      - ./data/raw:/docker-entrypoint-initdb.d:ro
      - mysql_data:/var/lib/mysql
volumes:
  mysql_data:
```

4. Start containers: docker-compose up -d

5. Wait for SQL import (check: docker-compose logs mysql)

Key Deliverables

- Taxonomy JSON with 200+ real estate terms across 8 categories
- Sample dataset: 500-1000 listing remarks extracted to CSV
- 50+ sample user queries with intent labels
- Data exploration notebook analyzing remark patterns
- Validation tests ensuring 30%+ taxonomy coverage
- GitHub repo initialized with teammates as collaborators

Implementation Guide

Create scripts/data_loading.py to extract sample data:

```
import mysql.connector
import pandas as pd

conn = mysql.connector.connect(
    host='localhost', user='root', password='root', database='real_estate')

query = """
SELECT L_ListingID, L_Address, L_City, L_Keyword2 as beds,
       LM_Dec_3 as baths, L_SystemPrice as price, L_Remarks as remarks
FROM   rets_property
WHERE  L_Remarks IS NOT NULL AND LENGTH(L_Remarks) > 50
ORDER BY RAND() LIMIT 1000
"""

df = pd.read_sql(query, conn)
```

```
df.to_csv('data/processed/listing_sample.csv', index=False)
conn.close()
```

Create scripts/taxonomy_builder.py for n-gram extraction:

```
import nltk
from collections import Counter
from nltk.util import ngrams

# Extract bigrams from remarks
all_text = ' '.join(df['remarks'].dropna().str.lower())
tokens = nltk.word_tokenize(all_text)
bigrams = list(ngrams(tokens, 2))
freq = Counter(bigrams)

# Top 200 bigrams become taxonomy seed
for bigram, count in freq.most_common(200):
    print(f"{' '.join(bigram)}: {count}")
```

Testing

Create tests/test_week1.py:

```
import pytest
import json

def test_taxonomy_loaded():
    with open('data/processed/taxonomy.json') as f:
        tax = json.load(f)
    assert len(tax['terms']) >= 200
    assert all('id' in t and 'term' in t for t in tax['terms'])

def test_sample_data_quality():
    df = pd.read_csv('data/processed/listing_sample.csv')
    assert len(df) >= 500
    assert df['remarks'].str.len().min() > 50
```


Week 2: Text Cleaning & Normalization

Overview

Build a text cleaning pipeline to standardize MLS remarks and queries. Handle abbreviations (br → bedroom), measurements (2,000 sqft → 2000 square feet), prices (450k → 450000), unicode issues, and HTML remnants.

Key Deliverables

- TextCleaner class with 6+ normalization methods
- Abbreviation dictionary with 30+ mappings
- Test suite with 40+ test cases covering edge cases
- Cleaned dataset generated from Week 1 sample
- Before/after examples showing improvements
- **NEW:** Data profiling report showing what needs cleaning (null rates, HTML presence, common abbreviations)

Implementation

Create scripts/text_cleaning.py:

```
import re

class TextCleaner:
    def __init__(self):
        self.abbrev_map = {
            'br': 'bedroom', 'ba': 'bathroom', 'sqft': 'square feet',
            'w/': 'with', 'w/o': 'without', 'mbr': 'master bedroom'
        }

    def clean_text(self, text):
        text = self.normalize_unicode(text)
        text = self.normalize_prices(text)
        text = self.normalize_measurements(text)
        text = self.expand_abbreviations(text)
        return text.strip()

    def normalize_prices(self, text):
        # 450k → 450000
        text = re.sub(r'(\d+)k', lambda m: str(int(m.group(1))*1000), text,
            flags=re.I)
        # 1.2m → 1200000
        text = re.sub(r'(\d+\.\d*)m', lambda m:
            str(int(float(m.group(1))*1000000)), text, flags=re.I)
        return text
```

Before cleaning, understand what needs cleaning:

```
def profile_column(self, df, column_name):
    """Analyze what's actually in L_Remarks"""
    return {
        'null_rate': df[column_name].isnull().mean(),
```

```

        'avg_length': df[column_name].str.len().mean(),
        'common_terms': self._extract_top_ngrams(df[column_name]),
        'price_mentions': df[column_name].str.contains(r'\$\d').sum(),
        'has_html': df[column_name].str.contains('<').sum(),
        'common_abbreviations': self._detect_abbreviations(df[column_name])
    }

# Use this to guide your cleaning strategy:
profile = cleaner.profile_column(df, 'remarks')
print(f"HTML tags found in {profile['has_html']} listings")
print(f"Common abbreviations: {profile['common_abbreviations']}")

```

Testing

```

def test_price_normalization():
    cleaner = TextCleaner()
    assert '450000' in cleaner.normalize_prices('priced at 450k')
    assert '1200000' in cleaner.normalize_prices('$1.2m home')

def test_profiling():
    profile = cleaner.profile_column(df, 'remarks')
    assert 'null_rate' in profile
    assert 'avg_length' in profile

```

Week 3: Named Entity Extraction

Overview

Extract structured entities from listing remarks: bedrooms, bathrooms, price, square footage, amenities. Start with regex rules, optionally train a custom spaCy NER model if time permits.

Key Deliverables

- EntityExtractor class with regex patterns for numeric entities
- Amenity detection using Week 1 taxonomy
- Labeled dataset: 200-300 remarks with entity spans
- Evaluation script calculating precision/recall/F1
- Target: 85%+ F1 score on test set
- Error analysis identifying top failure patterns

Implementation

Create scripts/entity_extractor.py:

```
import re

class EntityExtractor:
    def extract_bedrooms(self, text):
        patterns = [
            r'(\d+)\s*(?:bed|br|bedroom)s?',
            r'(\d+)bd'
        ]
        for pattern in patterns:
            match = re.search(pattern, text, re.I)
            if match:
                return int(match.group(1))
        return None

    def extract_price(self, text):
        # Assumes cleaned text from Week 2
        match = re.search(r'\$?(\d{5,})', text)
        return int(match.group(1)) if match else None

    def extract_all(self, text):
        return {
            'bedrooms': self.extract_bedrooms(text),
            'bathrooms': self.extract_bathrooms(text),
            'price': self.extract_price(text),
            'sqft': self.extract_sqft(text),
            'amenities': self.extract_amenities(text)
        }
```

Week 4: Query Parser → SQL

Overview

Parse natural language queries into structured filters and safe SQL WHERE clauses. Handle patterns like "3 bed under 700k in Irvine" → {beds=3, price_max=700000, city=Irvine}.

Key Deliverables

- QueryParser class handling 20+ query patterns
- Safe SQL generator using parameterized queries (NO string concat)
- Support for: price ranges, bed/bath min/max, amenities, negation
- Test suite with 50+ query examples
- 90%+ accuracy parsing Week 1 sample queries
- SQL injection protection verified
- **NEW:** Schema validation catching invalid cities, out-of-range values

Implementation

```
class QueryParser:
    def parse(self, query):
        filters = {}

        # Price patterns
        if match := re.search(r'under\s+\$(\d+)([km]?)', query, re.I):
            filters['price_max'] = self._parse_number(match.group(1),
match.group(2))

        # Bedroom patterns
        if match := re.search(r'(\d+)\s*\s*(?:bed|br)', query, re.I):
            filters['bedrooms_min' if '+' in match.group(0) else 'bedrooms'] =
int(match.group(1))

        return filters

    def to_sql(self, filters):
        conditions = []
        params = []

        if 'price_max' in filters:
            conditions.append('L_SystemPrice <= %s')
            params.append(filters['price_max'])

        if 'bedrooms' in filters:
            conditions.append('L_Keyword2 = %s')
            params.append(filters['bedrooms'])

        where_clause = ' AND '.join(conditions)
        return f"SELECT * FROM rets_property WHERE {where_clause}", params
```

Add validation before generating SQL:

```

class SchemaValidator:
    def __init__(self, schema_path='data/schema.json'):
        with open(schema_path) as f:
            self.schema = json.load(f)
            self.valid_cities = self._load_valid_cities()

    def validate_query(self, filters):
        errors = []

        # Check city exists in database
        if 'city' in filters:
            if filters['city'] not in self.valid_cities:
                errors.append(f"City '{filters['city']}' not found in
database")

        # Check price range
        if 'price_max' in filters:
            if filters['price_max'] < 100000 or filters['price_max'] >
100000000:
                errors.append(f"Price {filters['price_max']} outside typical
range")

        # Check bedroom count
        if 'bedrooms' in filters:
            if filters['bedrooms'] < 1 or filters['bedrooms'] > 10:
                errors.append(f"Bedroom count {filters['bedrooms']} seems
invalid")

        return len(errors) == 0, errors

# Usage:
parser = QueryParser()
validator = SchemaValidator()
filters = parser.parse("3 bed in Portland under 500k")
valid, errors = validator.validate_query(filters)
if not valid:
    print(f"Query validation errors: {errors}")
    # Return helpful message to user
else:
    sql, params = parser.to_sql(filters)

```

Week 5: Semantic Search with Embeddings

Overview

Implement embedding-based semantic search using sentence-transformers. Build FAISS index for fast similarity search. Compare semantic vs keyword matching quality.

Key Deliverables

- Sentence embeddings for all listing remarks (384 or 768 dims)
- FAISS index for efficient similarity search
- SemanticSearcher class with query embedding + retrieval
- Comparison study: semantic vs BM25 keyword search
- Latency < 100ms for 10k listings
- Relevance evaluation on 50 query-result pairs

Implementation

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

class SemanticSearcher:
    def __init__(self):
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.index = None
        self.listings = None

    def build_index(self, remarks_list):
        print(f"Encoding {len(remarks_list)} listings...")
        embeddings = self.model.encode(remarks_list)

        # Build FAISS index
        dim = embeddings.shape[1]
        self.index = faiss.IndexFlatIP(dim) # Inner product for cosine sim
        faiss.normalize_L2(embeddings)
        self.index.add(embeddings)
        self.listings = remarks_list

    def search(self, query, top_k=10):
        query_emb = self.model.encode([query])
        faiss.normalize_L2(query_emb)

        scores, indices = self.index.search(query_emb, top_k)
        results = [(self.listings[i], scores[0][j]) for j, i in
enumerate(indices[0])]
        return results
```

Week 6: Listing Signal Extraction

Overview

Extract comprehensive structured signals from each listing: amenities (pool, fireplace), condition (updated, new), financing (seller financing), location features. Combine Week 3 entities with taxonomy-based pattern matching.

Key Deliverables

- SignalExtractor class processing full listing records
- JSON output schema: entities + amenities + keywords
- Process entire rets_property table (save to processed/)
- Extraction accuracy: 90%+ for structured fields, 75%+ for free text
- Output suitable for search indexing and filtering

Implementation

```
class SignalExtractor:
    def __init__(self, taxonomy, entity_extractor):
        self.taxonomy = taxonomy
        self.extractor = entity_extractor

    def extract_signals(self, listing_record):
        remarks = listing_record.get('L_Remarks', '')

        # Get entities from Week 3
        entities = self.extractor.extract_all(remarks)

        # Get amenities from taxonomy
        amenities = self._match_amenities(remarks)

        # Detect signals
        return {
            'listing_id': listing_record['L_ListingID'],
            'entities': entities,
            'amenities': amenities,
            'condition_keywords': self._extract_condition(remarks),
            'financing_terms': self._extract_financing(remarks),
            'location_features': self._extract_location(remarks)
        }
```

Updated Week 7: Search Query Intent Classification

Overview

Classify real estate search queries into language-based intent categories such as browsing, researching, and high-intent inquiry. This module focuses on NLP-based query understanding using the wording and structure of the query itself — not behavioral buyer analytics or CRM conversion data.

Students will manually label search queries and train a lightweight text classification model to infer probable user intent from query language patterns.

Key Deliverables

- Labeled dataset: 200+ real estate search queries with intent labels
- QueryIntentClassifier using logistic regression or lightweight transformer models
- 80%+ accuracy on held-out test set
- Confidence scores for uncertain classifications
- Integration with Week 4 query parser for richer query understanding
- Documentation explaining limitations of language-based intent inference

Implementation

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
class QueryIntentClassifier:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(max_features=500)
        self.model = LogisticRegression()
        self.labels = [
            'browsing',
            'researching',
            'high_intent_inquiry'
        ]

    def train(self, queries, labels):
        X = self.vectorizer.fit_transform(queries)
        self.model.fit(X, labels)

    def predict(self, query):
        X = self.vectorizer.transform([query])

        probas = self.model.predict_proba(X)[0]

        intent = self.labels[probas.argmax()]
        confidence = probas.max()
```


return intent, confidence

Example Query Categories

Browsing

- show me homes in San Diego
- homes with pools in Irvine
- luxury homes in Beverly Hills

Researching

- condos near UC Irvine with low HOA
- areas in San Diego with low property taxes

High-Intent Inquiry

- move-in ready homes in San Diego under 1.2m
- homes available this weekend with open houses
- new listings in Irvine under 900k with seller financing

Example Usage

```
classifier = QueryIntentClassifier()
```

```
queries = [  
    "show me homes in San Diego",  
    "condos near UC Irvine with low HOA",  
    "move-in ready homes in San Diego under 1.2m"  
]
```

```
labels = [  
    "browsing",  
    "researching",  
    "high_intent_inquiry"  
]
```

```
classifier.train(queries, labels)
```

```
intent, confidence = classifier.predict(  
    "homes with open houses this weekend"  
)
```

```
print(intent, confidence)
```

Intent labels are inferred solely from the language of the search query and are intended for educational NLP classification purposes only.

Week 8: Listing Summarization

Overview

Generate concise 2-3 sentence summaries of listings for search results or email alerts. Implement extractive (select key sentences) and optionally abstractive (generate new text) approaches. Evaluate with ROUGE metrics.

Key Deliverables

- ListingSummarizer class with extractive method
- Optional: fine-tuned BART or T5 model for abstractive summaries
- ROUGE-L score > 0.4 on test set
- Human evaluation: 20 summaries rated by teammates
- Summaries include: beds/baths, price, top 2 features, location
- **NEW:** Answerability layer that explains why queries cannot be answered

Implementation

```
import nltk

class ListingSummarizer:
    def extractive_summary(self, remarks, entities, num_sentences=2):
        sentences = nltk.sent_tokenize(remarks)

        # Score sentences by entity mentions and position
        scores = []
        for i, sent in enumerate(sentences):
            score = 0
            # First sentence bonus
            if i == 0:
                score += 2
            # Entity mentions
            if str(entities.get('bedrooms', '')) in sent:
                score += 1
            if 'pool' in sent.lower():
                score += 1
            scores.append((score, sent))

        # Return top sentences
        top_sentences = sorted(scores, reverse=True)[:num_sentences]
        return ' '.join(s[1] for s in sorted(top_sentences, key=lambda x:
sentences.index(x[1])))
```

Add layer to determine if queries can be answered:

```
class AnswerabilityChecker:
    def __init__(self, taxonomy, schema_validator):
        self.taxonomy = taxonomy
        self.validator = schema_validator
        self.real_estate_keywords = ['house', 'home', 'bed', 'bath',
'property',
                                     'listing', 'price', 'sqft', 'pool',
'garage']
```

```

def check_pre_query(self, query):
    """Check BEFORE generating SQL"""
    query_lower = query.lower()

    # Check 1: Is this a real estate question?
    has_re_terms = any(kw in query_lower for kw in
self.real_estate_keywords)
    if not has_re_terms:
        return False, "This doesn't appear to be a real estate question"

    # Check 2: Does query reference valid data?
    # (Use Week 4's schema validator)
    filters = parser.parse(query)
    valid, errors = self.validator.validate_query(filters)
    if not valid:
        return False, f"Query references invalid data: {'; '.join(errors)}"

    return True, "Query is answerable"

def check_post_query(self, query, results_df):
    """Check AFTER executing SQL"""
    if len(results_df) == 0:
        return False, "No listings match your criteria"

    # Check for all-null results
    if results_df.isnull().all().all():
        return False, "Query returned no meaningful data"

    return True, "Results found"

# Usage:
checker = AnswerabilityChecker(taxonomy, validator)
can_answer, message = checker.check_pre_query(user_query)
if not can_answer:
    return {"error": message, "answerable": False}

# Execute query...
results = execute_query(sql)
can_answer, message = checker.check_post_query(user_query, results)
if not can_answer:
    return {"message": message, "results": []}

```

Week 9: Fair Housing Compliance

Overview

Implement Fair Housing Act compliance checking. Detect prohibited language related to protected classes (race, religion, familial status, etc.). Flag biased descriptions before publication.

Key Deliverables

- ComplianceChecker with pattern library for prohibited terms
- Multi-level warnings: error (must fix), warning (review), info
- 100% recall on test set of known violations (no false negatives)
- Precision > 80% (minimize false positives)
- Documentation explaining Fair Housing rules to team
- Integration example with listing submission workflow

Implementation

```
class ComplianceChecker:
    def __init__(self):
        self.prohibited_patterns = {
            'familial': ['no children', 'adults only', 'perfect for singles'],
            'disability': ['no wheelchairs', 'must be able-bodied'],
            'race': ['white neighborhood', 'ethnic', 'diverse area'],
            'religion': ['christian community', 'jewish neighborhood']
        }

    def check_listing(self, text):
        violations = []
        text_lower = text.lower()

        for category, patterns in self.prohibited_patterns.items():
            for pattern in patterns:
                if pattern in text_lower:
                    violations.append({
                        'category': category,
                        'pattern': pattern,
                        'severity': 'error',
                        'message': f'Prohibited language: {pattern} (Fair
Housing violation)'
                    })

        return {'compliant': len(violations) == 0, 'violations': violations}
```

Week 10: REST API with FastAPI

Overview

Build production-ready REST API exposing all NLP capabilities: search, entity extraction, query parsing, summarization, compliance checking. Add caching, rate limiting, logging.

Key Deliverables

- FastAPI application with 8+ endpoints
- Endpoints: /search, /parse-query, /extract-entities, /summarize, /check-compliance
- Request validation using Pydantic models
- Response caching (Redis or in-memory)
- Rate limiting (10 requests/second per IP)
- OpenAPI docs auto-generated
- Docker container for API deployment

Implementation

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Real Estate NLP API")

class SearchRequest(BaseModel):
    query: str
    top_k: int = 10

class SearchResponse(BaseModel):
    query: str
    results: list
    count: int

@app.post("/search", response_model=SearchResponse)
async def search_listings(request: SearchRequest):
    # Parse query
    filters = query_parser.parse(request.query)

    # Get semantic results
    semantic_results = semantic_searcher.search(request.query, request.top_k)

    # Combine with filters
    results = apply_filters(semantic_results, filters)

    return SearchResponse(
        query=request.query,
        results=results,
        count=len(results)
    )
```

Week 11: Product Integration Demo

Overview

Create end-to-end demo integrating all components. Build simple web interface showing complete user journey: search → filtered results → summaries → details. Demonstrate value of NLP enhancements over basic SQL search.

Key Deliverables

- Web UI (Streamlit or React) consuming Week 10 API
- Demo flow: natural language query → parsed filters → semantic results → summaries
- Side-by-side comparison: NLP search vs keyword-only search
- Metrics dashboard: query volume, latency, user satisfaction proxy
- Demo video (3-5 minutes) for presentation
- Deployment to free hosting (Render, Heroku, or Railway)

Implementation

```
import streamlit as st
import requests

st.title("Real Estate Intelligent Search")

query = st.text_input("What are you looking for?", "3 bed 2 bath under 700k in Irvine")

if st.button("Search"):
    response = requests.post("http://localhost:8000/search", json={"query": query})
    results = response.json()

    st.write(f"Found {results['count']} listings")

    for listing in results['results']:
        st.subheader(listing['address'])
        st.write(f"Price: ${listing['price']:,}")
        st.write(f"Summary: {listing['summary']}")
        st.write("---")
```

Week 12: Final Presentation & Defense

Overview

Prepare comprehensive presentation of full project. Demonstrate technical depth, business value, and lessons learned. Present to team and defend technical decisions.

Presentation Requirements

- Slides (15-20): problem, approach, architecture, results, learnings
- Live demo (5 min) showing key capabilities
- Metrics summary: accuracy, latency, coverage for each component
- Technical deep dive on 2-3 challenging aspects
- Discussion of production considerations and future work
- Q&A preparation for technical questions
- Individual contribution summary (if team project)

Final Deliverables

- GitHub repository with complete codebase
- README with setup instructions and architecture overview
- Technical documentation (API docs, data schemas)
- Test coverage report (aim for 70%+ coverage)
- Demo video and presentation slides
- Written report (5-10 pages) with metrics and analysis
- Deployment artifacts (Docker images, configs)
- Future work roadmap

Common Defense Questions

- Why did you choose [algorithm/model] over alternatives?
- How would you scale this to 10M listings?
- What were the biggest technical challenges?
- How do you handle edge cases in query parsing?
- What metrics matter most for this system?
- How would you monitor this in production?

Appendix A: Database Schema Reference

Key fields from `rets_property`:

- `L_ListingID`: Unique listing identifier
- `L_Address`: Property address
- `L_SystemPrice`: Current price
- `L_Keyword2`: Bedrooms
- `LM_Dec_3`: Bathrooms
- `LM_Int2_3`: Square footage
- `L_Remarks`: Agent description (main NLP target)
- `PoolPrivateYN`: Pool presence flag
- `FireplaceYN`: Fireplace flag
- `ViewYN`: View feature flag

Appendix B: Useful Libraries

- `nltk`: Tokenization, n-grams, basic NLP
- `spaCy`: NER, POS tagging, dependency parsing
- `sentence-transformers`: Sentence embeddings
- `faiss`: Vector similarity search
- `transformers`: Hugging Face models
- `fastapi`: REST API framework
- `pytest`: Testing framework
- `pandas`: Data manipulation
- `scikit-learn`: ML models and evaluation